P1874R0
Dynamic Initialization Order of Non-Local Variables in Modules

Draft Proposal, 2019-10-07

This version:
http://wg21.link/P1874r0

Author:
Michael Spencer (Apple)

Audience:
EWG, SG2

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The order of dynamic initialization of non-local variables with static storage duration (globals) imported from interface and header units is indeterminately sequenced relative to those of their importer. This has the potential to break code, including code which includes `<iostream>`, when moving to C++20. This paper explores what existing implementations do in this case, and defines an ordering as is currently done in both Clang and GCC.

Table of Contents

1 Effects of This Paper

2 The Problem

3 The Solution

3.1 The Catch

4 Implementation Units?

5 Ship Vehicle

6 Proposed Polls

7 Wording

7.1 [basic.start.dynamic]

§ 1. Effects of This Paper

> Note: In all of the examples in this paper it is assumed that `#include` translation does not occur.

Status Quo	This Paper

`import <iostream>;`
 	
`struct G {`
`  G() {`
`  std::cout << "Constructing\n";`
`  }`
`} g;`
`// ☠ std::Init may not have been initialized yet,`
`// so std::cout may not have been initialized yet.`
	`import <iostream>;`
	
	`struct G {`
	`  G() {`
	`  std::cout << "Constructing\n";`
	`  }`
	`} g;`
	`// ✅ std::Init has been initialized,`
	`// thus std::cout has been initialized.`

§ 2. The Problem

[basic.start.dynamic] states:

> Dynamic initialization of non-local variables `V` and `W` with static storage duration are ordered as follows:
>
> - If `V` and `W` have ordered initialization and `V` is defined before `W` within a single translation unit, the initialization of `V` is sequenced before the initialization of `W`.
> - If `V` has partially-ordered initialization, `W` does not have unordered initialization, and `V` is defined before `W` in every translation unit in which `W` is defined, then
>   - if the program starts a thread ([intro.multithread]) other than the main thread ([basic.start.main]), the initialization of `V` strongly happens before the initialization of `W`;
>   - otherwise, the initialization of `V` is sequenced before the initialization of `W`.
> - Otherwise, if the program starts a thread other than the main thread before either `V` or `W` is initialized, it is unspecified in which threads the initializations of `V` and `W` occur; the initializations are unsequenced if they occur in the same thread.
> - Otherwise, the initializations of `V` and `W` are indeterminately sequenced.

With textual `#include`s the first rule holds for global initaliziers, but this breaks when moving to `import`, either explicitly or via `#include` translation. This happens because header units are their own translation units, and thus fall into the last rule. A similar problem exists with module interface units which transitively `#include` or `import` such a header.

§ 3. The Solution

Clang already ran into this issue and has a simple fix. Run all global initaliziers from translation units which are transitively imported when they are imported.

```
// H1.h
inline int a = init();

// H2.h
inline int b = init();

// TU.cpp
int c = init();
import "H1.h";
import "H2.h";
```

Clang modules tried to mimic includes as closely as possible, thus in the above example Clang will initialize `c` first, and then `a` before `b`. If you flip the order of the imports, then it would initialize `b` before `a`. While this is a reasonable model for Clang modules, which was intended to mimic `#include`s, it is a poor fit for C++20 modules, as it violates the rule that the order of imports doesn't matter.

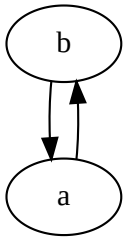
This paper explores a simpler user model based on dependency order. All global initaliziers from translation units which are transitively imported are run before any globals in the translation unit which imports them. For the above example it would mean that the initialization of `a` and `b` is sequenced before the initialization of `c`, but that the initializations of `a` and `b` are indeterminately sequenced.

§ 3.1. The Catch

Initializers of inline variable `V` and variable `W` are ordered when "`V` is defined before `W` in every translation unit in which `W` is defined." If the above solution is applied blindly, then cycles can be created.

```
// H1.h
int external(int);

// H2.h
#include "H1.h"
inline int a = external(0);

// H3.h
#include "H1.h"
inline int b = external(1);

// M1.cppm
module;
#include "H2.h"
export module M1;
import "H3.h";

// M2.cppm
module;
#include "H3.h"
export module M2;
import "H2.h";
```

This example has the following sequenced before graph:



M1 has an interface dependency on the the header unit "H3.h", and contains a definition of `b`, thus `b`s initialization is sequenced before `a`'s initialization. However; M2 has an interface dependency on the header unit "M2.h" and contains a definition of `a`, thus `a`s initialization is sequenced before `b`'s initialization. This is a contradiction.

The ordering guarantees will need to be weakened for this case.

§ 4. Implementation Units?

If we're defining an ordering for interfaces, why not also define one for module implementation units and solve the init order problem once and for all? While there are plausible ways to define an ordering for implementation units, there are several reasons why this direction was not chosen at this time:

- At best it can only order between different modules, not between implementation units of a single module.
- Implementation units can form circular dependencies between modules.
- It has not been implemented, and has an unknown runtime impact, potentially requring adding a global constructor to every module.
- It is not required to fix the `<iostream>` issue, and thus out of scope for C++20.

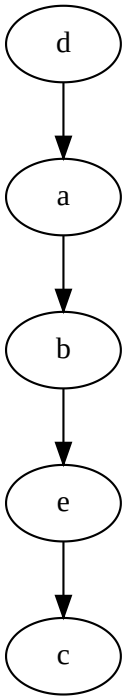
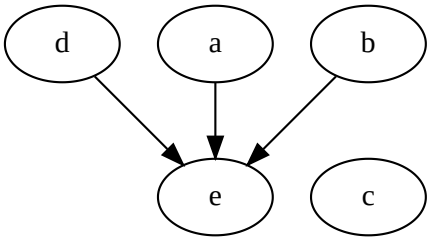
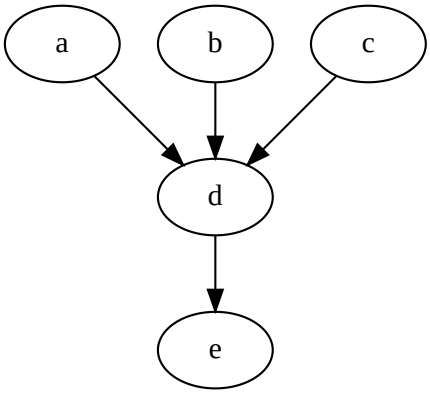
§ 5. Ship Vehicle

This paper is targeting C++20 as the standard library is subtly broken without it.

§ 6. Proposed Polls

1. Pick a model. In the graphs that follow a directed arrow represents a sequenced before relationship. Initializers with no forward path between them are indeterminately sequenced.

```
// H1.h
int a = init();

// H2.h
int b = init();

// H3.h
int c = init();

// TU.cpp
int d = init();
import "H1.h";
import "H2.h";
int e = init();
import "H3.h";
```

Current Clang Model	Relaxed Clang Model	Dependency Order Model
		

2. Forward P1874 to CWG for C++20.

§ 7. Wording

The following wording assumes that the issue with *interface dependency* not applying to header units is fixed.

Wording Note

This is a wording note. Its purpose is to help clarify the intent of the wording in this document to the members of the committee. It is not an instruction to the editor.

§ 7.1. [basic.start.dynamic]

[2] Dynamic initialization of non-local variables `V` and `W` with static storage duration are ordered as follows:

- If `V` and `W` have ordered initialization and `V` is defined before `W` within a single translation unit , or `V` is in a translation unit on which `W`'s translation unit has an *interface dependency* , the initialization of `V` is sequenced before the initialization of `W`.
- If `V` has partially-ordered initialization, `W` does not have unordered initialization, and `V` is ~~defined before~~ reachable from `W` in every translation unit in which `W` is defined, then

  Wording Note

  I believe that reachability is the correct thing to use in this case. It covers both "defined before" and interface dependency. It is not used for the previous case because it is slightly weaker, as it matters where the import occurs.

  - if the program starts a thread ([intro.multithread]) other than the main thread ([basic.start.main]), the initialization of `V` strongly happens before the initialization of `W`;
  - otherwise, the initialization of `V` is sequenced before the initialization of `W`.
- Otherwise, if the program starts a thread other than the main thread before either `V` or `W` is initialized, it is unspecified in which threads the initializations of `V` and `W` occur; the initializations are unsequenced if they occur in the same thread.
- Otherwise, the initializations of `V` and `W` are indeterminately sequenced.

[*Note:* This definition permits initialization of a sequence of ordered variables concurrently with another sequence. — *end note*]